EE4705 Robotics — Project 2

Collision-Free Path Planning for Robot Manipulators with STOMP

Group 9 — Report and Code

Joshua Lim · Hu Jiamo · Dexter

National University of Singapore, AY 2026/27

Submitted 10 November 2026

We implement the Stochastic Trajectory Optimization for Motion Planning (STOMP) algorithm in MATLAB and use it on two 7-DOF manipulators. Task 1 completes the provided sample code for the Kinova Gen3. Tasks 2–5 use the Franka Emika Panda: a Task-1-like scene (Task 2), forward kinematics by the Product of Exponentials instead of getTransform (Task 3; identical results to 1e-15, 35x faster FK and 14x faster STOMP iterations), three multi-obstacle scenarios that are all solved for every one of 10 random seeds (Task 4), and an end-effector orientation constraint that keeps the gripper y-axis upright within 1.9° instead of 36.8° (Task 5). Beyond the tasks we fix ten bugs of the sample code, make STOMP respect joint limits, add sample reuse (fewer iterations), handle the ±π joint-angle discontinuity, fit the collision spheres to the robot meshes, provide an interactive GUI, and verify every result with a mesh-based collision check at and between the waypoints.

Contents

1 Introduction and code organisation 3
2 STOMP formulation and implementation 3
3 Task 1 — Completing the sample code (Kinova Gen3) 4
4 Task 2 — A different manipulator: Franka Emika Panda 5
4.1 Choice of robot and model 5
4.2 Scene 5
4.3 Collision spheres fitted to the robot meshes 5
5 Task 3 — Forward kinematics by the Product of Exponentials 6
5.1 Determining the screw axes 6
5.2 Efficient implementation 7
5.3 Effect on planning 8
6 Task 4 — Custom multi-obstacle scenarios 8
7 Task 5 — End-effector orientation constraint 9
7.1 Constraint cost and scene 9
7.2 Choice of the weight 9
7.3 Result with and without the constraint 10
8 Improvements and additional features 11
8.1 Bugs fixed in the provided code 11
8.2 STOMP extensions: joint limits and sample reuse 12
8.3 Convergence patience 12
8.4 Joint-angle discontinuity at ±π 13
8.5 Further practical features 13
9 Limitations 14
10 Contributions 14
11 AI declaration 15
References 15

1 Introduction and code organisation

STOMP [1] optimises a whole joint-space trajectory at once: it perturbs the current trajectory with smooth noise, scores every noisy trajectory with a cost that penalises collisions (and, in Task 5, a violated orientation), and moves the trajectory towards the low-cost perturbations. It needs no gradient of the cost, which suits the non-smooth collision cost of a voxelised world.

The submission is one code base. `common/` holds the shared planner, kinematics, world model and plotting functions; each task has a folder with one entry script (`main_task1.m` … `main_task5.m`), plus `Bonus_STOMPExtensions/main_bonus.m` for the additional experiments and `Videos/make_videos.m` for the presentation videos, and `GUI/stomp_gui.m` is an interactive front end. Every script sets its own path and seeds the random number generator, so a run reproduces our numbers exactly; the README lists the toolboxes and run times. The `results/` folders in the zip contain the figures, tables and videos of the final run. All parameters live in `common/config/defaultConfig.m` (Table 1) and the robot of Tasks 2–5 in `robotConfig.m`.

2 STOMP formulation and implementation

A trajectory is a matrix $\theta \in \mathbb{R}^{n\times N}$ of $N = 20$ waypoints of the $n = 7$ joint angles; the start $\theta_1 = q_0$ and the goal $\theta_N = q_T$ are fixed (inverse kinematics of the given end-effector poses). The robot is approximated by spheres placed along the straight segments between consecutive joint frames, and the world by a 2 cm voxel grid with its signed Euclidean distance transform $d(x)$ (`bwdist`). The local cost of waypoint i and the total cost are

$$S(\theta_i) = w_{\text{obs}} q_o(\theta_i) + w_{\text{con}} q_c(\theta_i), \qquad Q(\theta) = \sum_i S(\theta_i) + \frac{1}{2}\sum_j \theta_j^T R \theta_j \tag{1}$$

with the obstacle cost of Eq. (13) of [1], summed over the spheres b of the arm (radius r_b, centre speed $\left\|\dot{x}_b\right\|$ from finite differences) and the safety margin ε:

$$q_o(\theta_i) = \sum_b \max(\varepsilon + r_b - d(x_b), 0)\|\dot{x}_b\| \tag{2}$$

$R = A^T A$ is the finite-difference acceleration matrix. Each iteration draws $K = 20$ noisy trajectories $\theta + \varepsilon_k$ with $\varepsilon_{k,j} \sim N(0, \Sigma)$ per joint, $\Sigma \propto R^{-1}$ (smooth noise), evaluates S for every sample and waypoint, converts the costs into probabilities per waypoint (Eq. (11) of [1]) and updates θ with the probability-weighted noise, smoothed by M (R^{-1} normalised per column):

$$P(\theta_{k,i}) = \frac{\exp(-h\tilde{S}_{k,i})}{\sum_l \exp(-h\tilde{S}_{l,i})}, \qquad \tilde{S}_{k,i} = \frac{S_{k,i} - \min_l S_{l,i}}{\max_l S_{l,i} - \min_l S_{l,i}} \tag{3}$$

$$\delta\theta_i = \sum_k P(\theta_{k,i})\varepsilon_{k,i}, \qquad \theta \leftarrow \theta + M\delta\theta \tag{4}$$

The loop stops when $|Q_{\text{new}} - Q_{\text{old}}| < 0.1$ for three consecutive iterations (a single iteration in the original code, where one small change can be a coincidence of the random sampling) or at the iteration cap (Section 8.3). A planned trajectory is accepted only if MATLAB's mesh-based `checkCollision` finds no contact at any waypoint **and** at nine joint-space sub-steps between every pair of waypoints (the robot moves continuously); this ground truth is independent of the sphere model that STOMP optimises.

Table 1. STOMP parameters (common/config/defaultConfig.m, robotConfig.m).

Parameter	Value	Meaning
N, K	20, 20	waypoints (incl. start/goal), noisy samples per iteration
h	10	sensitivity of the probabilities, Eq. (3)
Σ scale	1.5	exploration noise $\Sigma = 1.5R^{-1}/\mathrm{sum}(R^{-1})$
w_{obs}	1000	weight of the obstacle cost
ε	0.05 m (Kinova), 0.02 m (Panda)	safety margin in Eq. (2) (Panda: see Section 4.3)
sphere radius	0.05 m (Kinova), 0.103–0.155 m (Panda)	Panda: fitted per link to the meshes
stop rule	$\lvert\Delta Q\rvert < 0.1$ three times	iteration cap 50 (Tasks 1–3), 200 (Tasks 4–5)
extensions	joint limits, 5 reused samples	Section 8.2

3 Task 1 — Completing the sample code (Kinova Gen3)

Task 1 runs with `Task1_Kinova/main_task1.m`, which replaces the provided `KINOVA_STOMP_Path_Planning.m`; the provided files are kept unmodified in `original_code/`. The provided script was turned into functions (`runSTOMP` replaces `helperSTOMP`) and the five incomplete files were completed:

- `runSTOMP`: the main loop of Section 2 — sampling, local costs of all samples, probabilities, probability-weighted update smoothed by M, total cost and the stopping rule.
- `updateJointsWorldPosition`: positions and poses of all joint frames and the end-effector. `getTransform` is called with the configuration vector directly (`DataFormat` column) instead of building a configuration struct on every call as the sample code did.
- `stompSamples`: per joint, K samples of the interior waypoints from $N(0, \Sigma)$ with `mvnrnd`, padded with zeros at the fixed start and goal.
- `stompDTheta`: $\delta\theta$ of Eq. (4) for every joint and waypoint.
- `stompObstacleCost`: Eq. (2) with the signed distance looked up in the voxel grid.

While completing them we found and fixed several bugs of the sample code (Section 8.1), e.g. samples leaving the voxel world got zero obstacle cost and therefore looked like the best samples. With the original scene (box of 16 cm in the way, start and goal as given), the straight-line initial trajectory collides at 8 of 20 waypoints; STOMP finds a collision-free trajectory in 23 iterations (≈ 0.49 s per iteration with getTransform). Figure 1 overlays the intermediate configurations; the animation plays at the end of `main_task1.m`. This run uses our extended STOMP (joint-limit projection, shortest-angle initialisation, sample reuse) and the patience-3 convergence rule (Sections 8.2 and 8.3).

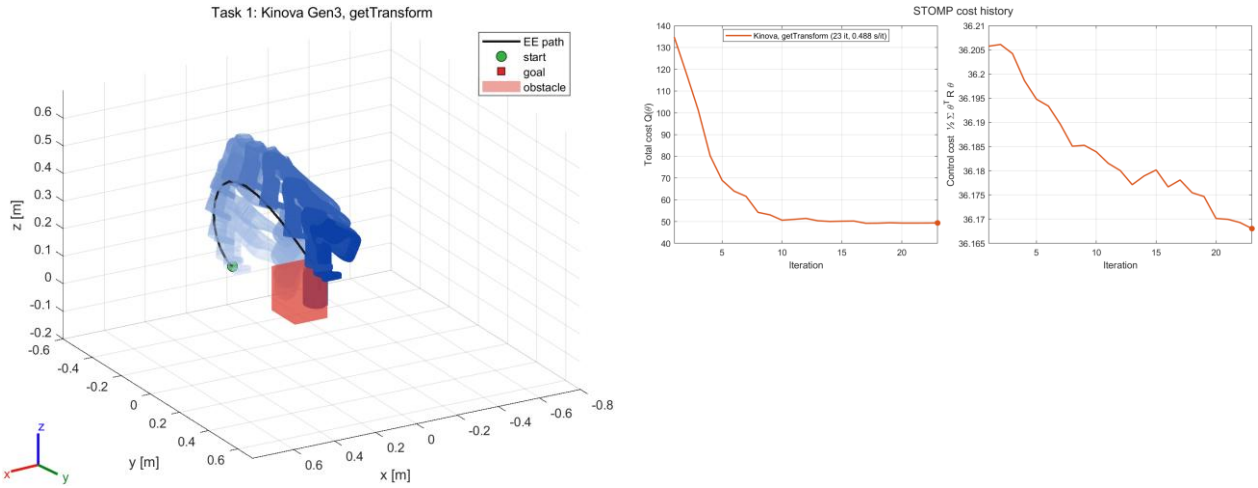


Figure 1. Task 1, Kinova Gen3: planned trajectory (light = start, dark = goal, black = end-effector path) and cost history.

4 Task 2 — A different manipulator: Franka Emika Panda

4.1 Choice of robot and model

We chose the Franka Emika Panda (`loadrobot('frankaEmikaPanda')`, URDF from franka_ros 0.6.0). It has seven revolute joints like the Kinova but a different geometry that the planner and the PoE code must handle: no continuous joints and tight limits (joint 4 $\in [-3.07, -0.07]$ rad, joint 6 $\in [-0.02, 3.75]$ rad), coincident frames (link 1/2 and link 5/6), 8.25 cm elbow offsets, and a gripper with two prismatic finger joints. `buildRobotModel` keeps only the movable joints of the base-to-end-effector chain in configuration order, freezes the finger joints (they are not planned), and adds a virtual tool-centre point `panda_tcp` 0.1034 m along the hand z-axis, matching the fingertips of the MATLAB model (the finger meshes extend ~0.11 m below the flange), so that task poses refer to the grasp point.

4.2 Scene

The scene follows Task 1: same start pose and a 16 cm box. As the handout warns, the Task-1 goal does not suit the Panda; the goal was moved to [0.30, 0.60, 0.45] m (tool down) where IK succeeds with clearance, and the box was placed on the joint-space straight line between start and goal (waypoint 12), so that the initial trajectory collides (7 of 20 waypoints). STOMP plans a collision-free trajectory in 13 iterations (≈ 0.5 s per iteration with getTransform; Figure 3). This run uses our extended STOMP (joint-limit projection, shortest-angle initialisation, sample reuse) and the patience-3 convergence rule.

4.3 Collision spheres fitted to the robot meshes

The first version used 5 cm spheres for every link, as for the Kinova. Drawing them over the Panda meshes showed that they lie inside the links: the Panda links are thick and the joint frames are offset from the link axes, so the median mesh distance to the sphere segments is 6–10 cm. Only the 5 cm safety margin kept the arm away from obstacles. We therefore fit one radius per segment (`fitSphereRadii`): for 200 random configurations inside the joint limits, every vertex of every collision mesh (the meshes `checkCollision` uses) is assigned to its nearest sphere segment, and the radius is the 99th percentile of these distances (Table 2). The spheres now contain 99 % of the robot, and the safety margin ε is a real 2 cm margin. Figure 2 shows the coverage; all Panda scenes remain solvable (Sections 6–8).

Table 2. Collision-sphere radii fitted to the Panda collision meshes (200 random configurations; segment k runs from frame $k-1$ to frame k). Zero-length segments (coincident frames) take the previous radius. robotConfig uses these values rounded up to mm.

Segment ends at	Mesh vertices	Median distance [m]	Radius = 99th pct. [m]	Max [m]
panda_link1	296	0.098	0.155	0.155
panda_link2	0	0.000	0.155	0.000
panda_link3	217	0.055	0.129	0.129
panda_link4	86	0.098	0.111	0.117
panda_link5	292	0.093	0.129	0.152
panda_link6	0	0.000	0.129	0.000
panda_link7	102	0.081	0.130	0.153
panda_tcp	211	0.081	0.103	0.153

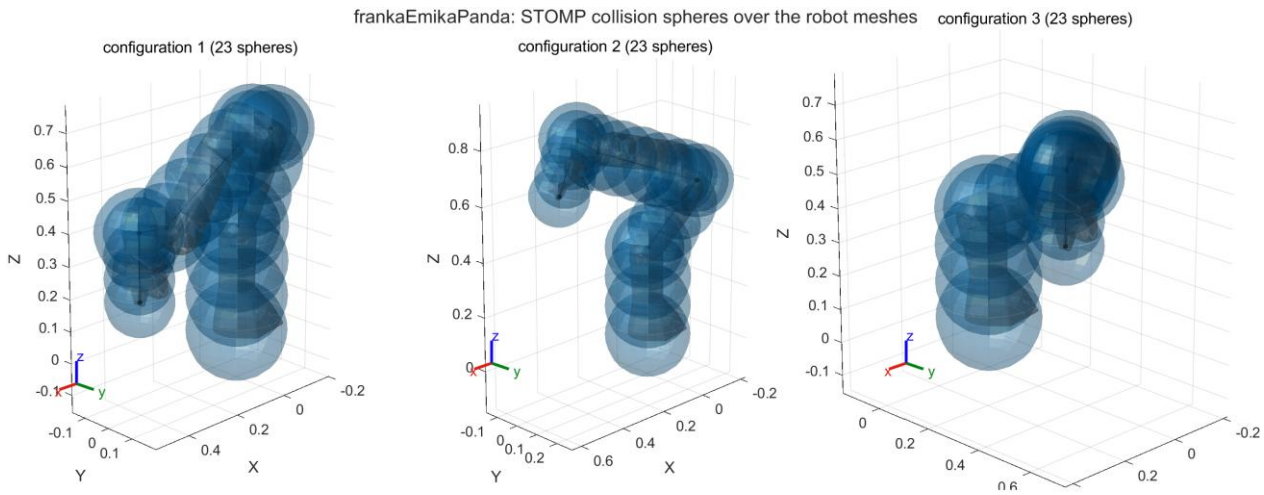


Figure 2. Fitted collision spheres over the Panda meshes at the start, middle and goal waypoint of Task 2.

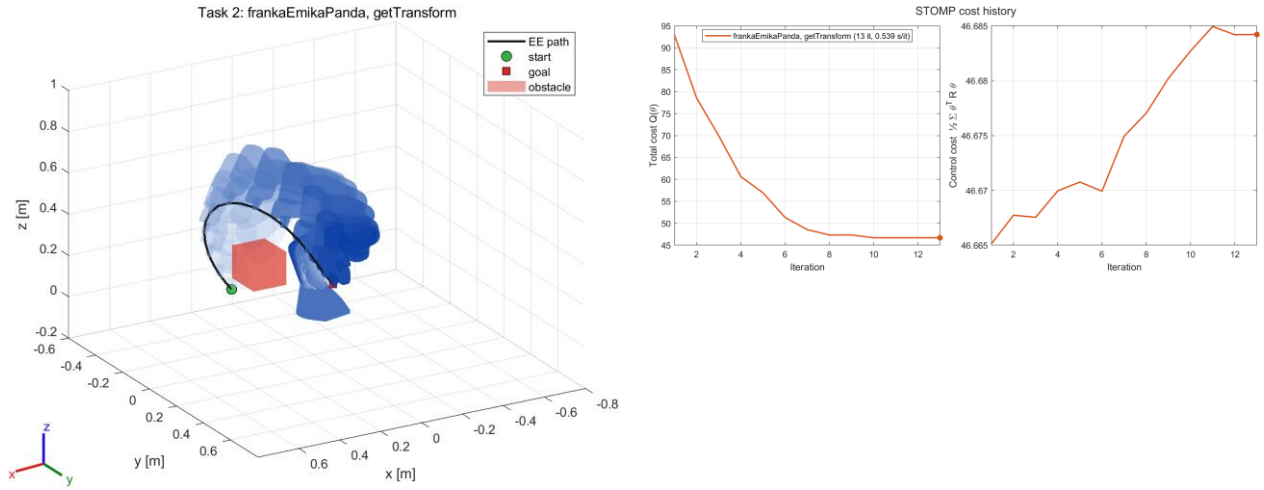


Figure 3. Task 2, Panda: planned trajectory and cost history.

5 Task 3 — Forward kinematics by the Product of Exponentials

5.1 Determining the screw axes

With all joint angles zero as the reference configuration, the pose of frame k is

$$T_k(\theta) = e^{[S_1]\theta_1} e^{[S_2]\theta_2} \cdots e^{[S_k]\theta_k} M_k \tag{5}$$

where $S_i = (\omega_i, v_i)$ is the space-frame screw axis of joint i and M_k the pose of frame k at the reference configuration. The screw axes are computed **once** (in `buildRobotModel`) from the robot description, as in the planar car example of the lecture:

- ω_i: the joint axis (`JointAxis`, given in the joint frame — not in the base frame, cf. the URDF `<axis>`) rotated into the base frame with the joint pose at the reference configuration (`getTransform` of the child body times the inverse `ChildToJointTransform`).
- q_i: the origin of that joint frame, a point on the axis; $v_i = -\omega_i \times q_i$. Prismatic joints would give $S_i = (0, \omega_i)$.
- M_k = `getTransform` of frame k at the reference configuration.

We use the zero configuration and not `homeConfiguration` (the Panda home pose has $\theta_4 = -1.57$), so that Eq. (5) holds with θ measured from zero; the zero pose lies outside the limits of joints 4 and 6 but is only a reference, the planner never goes there. As a cross-check (method 2 of the handout), each screw axis was also recovered with the matrix logarithm: moving only joint i by $\alpha = 0.7$ rad gives $T_i(\alpha) = e^{[S_i]\alpha} M_i$, so $[S_i] = \log(T_i(\alpha) M_i^{-1})/\alpha$. Both methods agree to 4.4e-16 for both robots. Table 3 lists the Panda axes next to the URDF data; Figure 4 draws them. For the Kinova, the table and figure in the results show $\omega_1 = (0,0,-1)$: the

URDF rotates joint 1 by rpy $= (\pi, 0,0)$, so its z-axis points down — exactly why the URDF axis cannot be used in the base frame directly.

Table 3. Space-frame screw axes of the Panda at the zero configuration (screw_axes_frankaEmikaPanda.csv; the URDF origin is relative to the parent link).

Joint	ω_i	q_i (closest axis point) [m]	$v_i = -\omega_i \times q_i$	URDF origin xyz
joint1	(0, 0, 1)	(0, 0, 0)	(0, 0, 0)	(0, 0, 0.333)
joint2	(0, 1, 0)	(0, 0, 0.3330)	(-0.3330, 0, 0)	(0, 0, 0)
joint3	(0, 0, 1)	(0, 0, 0)	(0, 0, 0)	(0, -0.316, 0)
joint4	(0, -1, 0)	(0.0825, 0, 0.6490)	(0.6490, 0, -0.0825)	(0.0825, 0, 0)
joint5	(0, 0, 1)	(0, 0, 0)	(0, 0, 0)	(-0.0825, 0.384, 0)
joint6	(0, -1, 0)	(0, 0, 1.0330)	(1.0330, 0, 0)	(0, 0, 0)
joint7	(0, 0, -1)	(0.0880, 0, 0)	(0, 0.0880, 0)	(0.088, 0, 0)

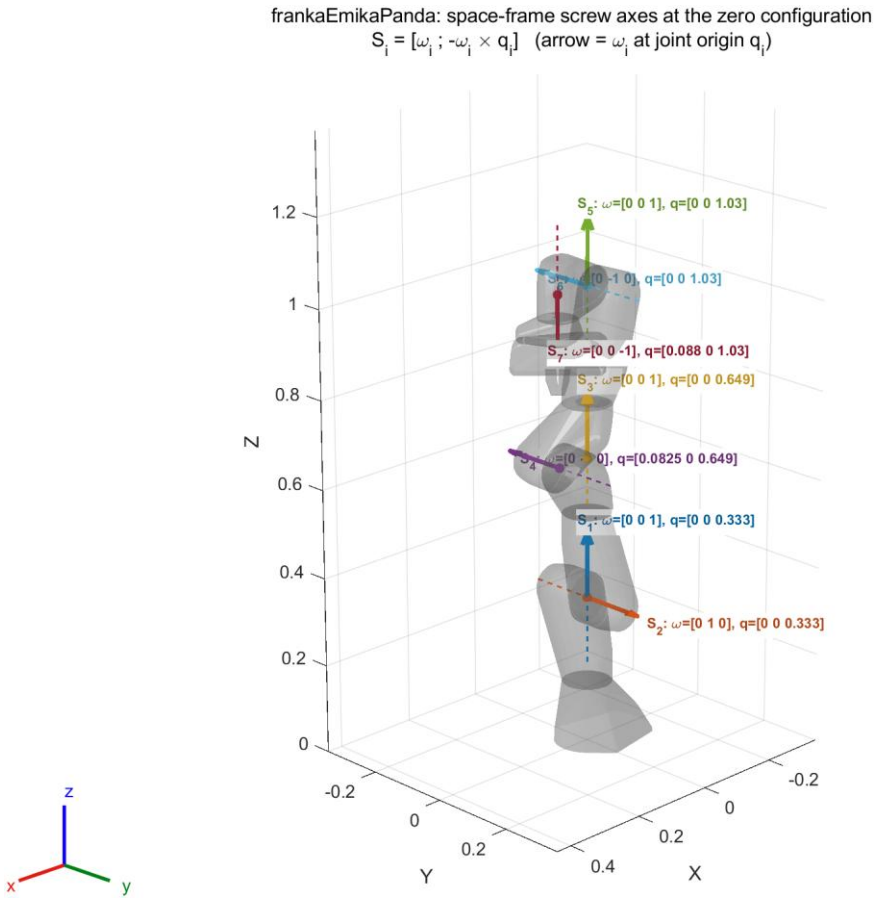


Figure 4. Screw axes of the Panda drawn at the zero configuration (arrow = ω_i at the joint origin q_i); the Kinova version is task3_screw_axes_kinovaGen3.png in the results.

5.2 Efficient implementation

The planner needs **every** joint frame (for the collision spheres), not only the end-effector. `fkPoE` therefore forms the product of Eq. (5) once from left to right and reads off T_k after the k-th factor: one exponential and one matrix product per joint, $O(n)$ instead of $O(n^2)$ for n separate end-effector-style calls. Each exponential uses the closed form (Assignment 2), $R = I + \sin\theta[\omega] + (1 - \cos\theta)[\omega]^2$, $p = (I\theta + (1 - \cos\theta)[\omega] + (\theta - \sin\theta)[\omega]^2)v$, instead of the general `expm`. The screw axes are computed once per robot, not per call.

Table 4. PoE vs getTransform over all frames of 500 random configurations within the joint limits; time per call to compute all frames (median of timeit). "expm per frame" is a naive PoE that recomputes the product with expm for every frame.

Robot	Max position error [m]	Max rotation error	getTransform [µs]	PoE [µs]	expm per frame [µs]	Speed-up
Panda	8.6e-16	1.1e-15	1274	36.5	563	35x
Kinova	1.0e-15	1.8e-15	780	25.1	335	31x

5.3 Effect on planning

Planning the Task-2 scene with the same seed gives the identical trajectory with both FK methods (same number of iterations, 13, and the same final cost 46.684), which confirms the implementation end to end. The mean STOMP iteration takes 527 ms with getTransform and 38 ms with PoE (13.9x faster), because FK of all frames is evaluated K·N = 400 times per iteration. Tasks 3–5 and the additional experiments use PoE; this is what makes the multi-seed studies of Sections 6–8 affordable.

6 Task 4 — Custom multi-obstacle scenarios

`buildVoxelWorld` was extended from one cube to any list of obstacles: boxes (optionally rotated by roll-pitch-yaw), spheres and upright cylinders. Curved and rotated shapes are voxelised by testing the voxel centres, padded by half a voxel so that thin shapes (a 3 cm post) are not under-sampled; obstacles that extend beyond the world are clamped instead of making `sub2ind` fail. The same obstacles are created as collision objects for the mesh-based check. The three scenarios (Figures 5 and 6) are designed so that the joint-space straight line collides and the robot has to make a real detour:

- **S1 — over the wall:** start and goal on either side of a 25 cm wall, tool down at 12 cm height; a post blocks the short way round the inner end, so the arm must lift the gripper over the wall.
- **S2 — under the shelf:** start above a 70 cm-wide shelf plate, goal under it with the tool horizontal; the arm must reach round the plate edge between the two legs while re-orienting the tool.
- **S3 — around the column:** a large re-orientation from in front of the robot to its left side past a 70 cm column, a ball on the floor and a panel rotated by 30°.

Table 5. Task 4 results (task4_summary.csv). Iterations/time: showcase run with seed 0. Robustness: STOMP from the same start and goal with seeds 1–10, success = no collision at or between the waypoints. Clearance: smallest distance between a collision sphere and an obstacle along the plan.

Scenario	Obstacles	Initial guess colliding	Iterations	Time [s]	Collision-free over 10 seeds	Median iterations	Min. sphere clearance [cm]
S1 Over the wall	2 (box, cylinder)	10 / 20	32	1.2	10 / 10	44	5.8
S2 Under the shelf	3 (box, cylinder)	6 / 20	20	0.6	10 / 10	23	1.0
S3 Around the column	3 (box, cylinder, sphere)	5 / 20	94	3.2	10 / 10	82	-0.1

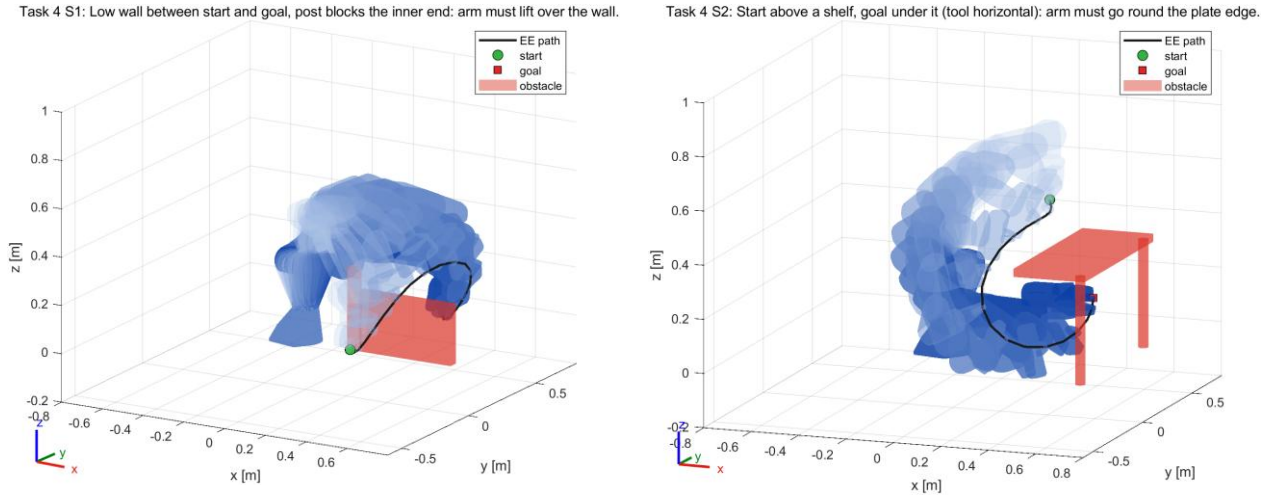


Figure 5. Task 4 scenarios S1 (left) and S2 (right): planned trajectories.

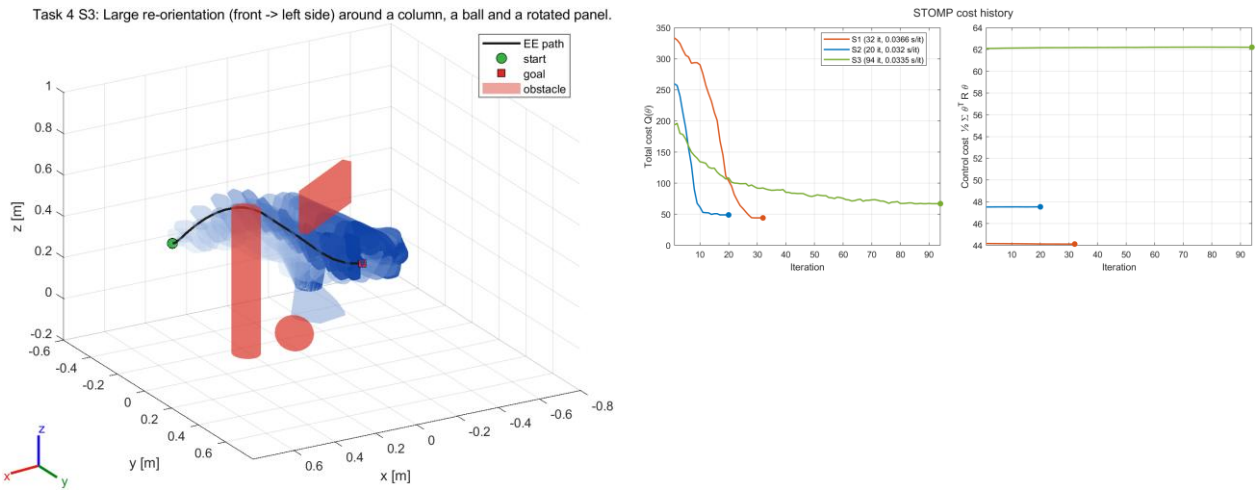


Figure 6. Task 4 scenario S3 (left) and the cost histories of the three scenarios (right).

S3 is the hardest scenario for the planner (most iterations over the seeds); the plan of S3 passes closest to the obstacles (one collision sphere reaches 0.1 cm into the voxelised obstacle — below the 2 cm voxel size and inside the 2 cm safety margin; the mesh-based check at and between the waypoints confirms that the robot itself stays clear). The iteration cap was raised from 50 to 200 for these scenes because the detours are longer than in Task 1; with 50 iterations a few waypoints were still in collision.

7 Task 5 — End-effector orientation constraint

7.1 Constraint cost and scene

Task 5 keeps the gripper y-axis upright ("carrying a cup of coffee"). Following option 2 of the handout, the constraint cost of a waypoint is the ℓ1 norm of the difference between the end-effector y-axis in the world frame, $\hat{y}_{sb}$ (second column of the rotation part of T_{sb}, available from FK at no cost), and the world z-axis:

$$q_c(\theta_i) = \|\hat{y}_{sb}(\theta_i) - \hat{z}_s\|_1 = \sum_{j=1}^{3} |\hat{y}_{sb,j} - \hat{z}_{s,j}| \tag{6}$$

It enters $S(\theta_i)$ with the weight w_{con} (Eq. (1)). Only the axis direction is constrained, the gripper may still turn about the vertical. As the handout suggests, start and goal are themselves upright: both are at 0.5 m from the base axis (azimuth −35° and 60°) with the tool pointing radially outward and its y-axis vertical, and a 16 cm box on the arc between them forces a detour. Our first version used a height of 0.30 m, which the Panda cannot reach with this orientation (IK stopped 2.8 cm away with joint 5 at its limit, even with 30 random restarts); at 0.20 m both poses are reached exactly, and the box was lowered with them. The initial trajectory collides at 6 of 20 waypoints.

7.2 Choice of the weight

STOMP normalises the local costs per waypoint before the probabilities (Eq. (3)), so only the ratio of constraint and obstacle cost matters. One run per weight can mislead because STOMP is stochastic; each weight was therefore run with 5 seeds (Table 6, Figure 7). The rule picks the smallest positive weight that is collision-free for every seed with a median worst-case tilt below 5°. Small weights already keep the tilt around 1°, while large weights (300, 1000, 3000) make the constraint dominate the obstacle term and collisions reappear — the trade-off the weight controls.

Table 6. Task 5 weight sweep, 5 seeds per weight (task5_weight_sweep.csv). $w_{\mathrm{con}} = 0$ is the planner without the constraint.

w_{con}	Collision-free	Median max tilt [°]	Worst max tilt [°]	Median mean tilt [°]	Median iterations
0	5 / 5	38.00	39.96	19.80	24
10	5 / 5	1.26	1.66	0.49	173
30	5 / 5	0.92	1.10	0.37	200
100	5 / 5	1.32	1.98	0.42	200
300	4 / 5	0.85	0.97	0.38	200
1000	0 / 5	0.76	0.84	0.32	200
3000	0 / 5	0.71	0.98	0.30	200

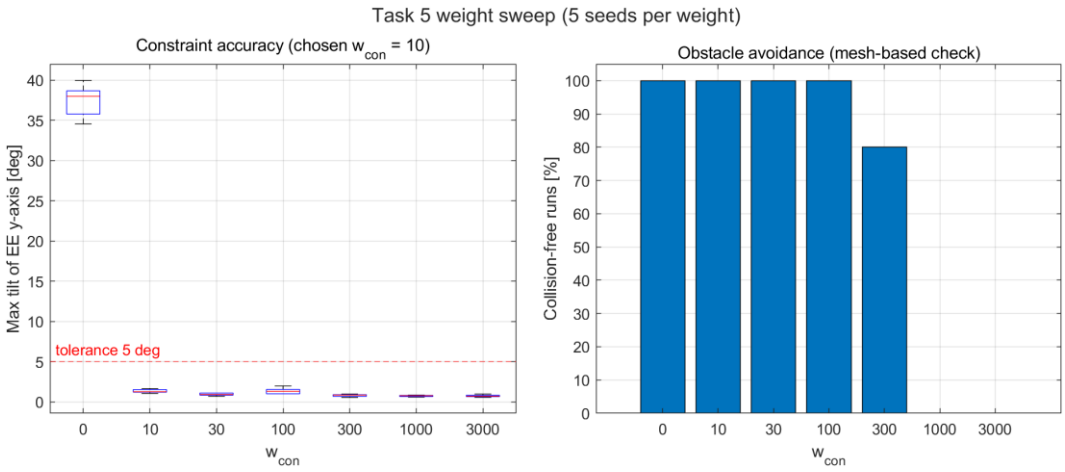


Figure 7. Weight sweep: spread of the maximum tilt over the seeds (left) and collision-free runs (right).

7.3 Result with and without the constraint

With the chosen weight $w_{\mathrm{con}} = 10$ and the same seed as the unconstrained run, the maximum tilt of the gripper y-axis drops from 36.8° to 1.9° and the mean tilt from 19.1° to 0.8° (Table 7, Figures 8 and 9); both trajectories are collision-free. Without the constraint the planner tilts the gripper by up to 37° in the middle of the detour, where it is free to do so. The price of the constraint is more iterations (27 → 124): the planner has to find a detour that is also upright, a much smaller set of trajectories. The constraint is a soft cost, so the tilt is small but not exactly zero.

Table 7. Task 5 before / after (task5_comparison.csv).

Run	Max tilt [°]	Mean tilt [°]	EE path [m]	Iterations	Time [s]	Collision
without constraint	36.78	19.13	0.839	27	1.0	no
with constraint	1.92	0.82	0.821	124	4.6	no

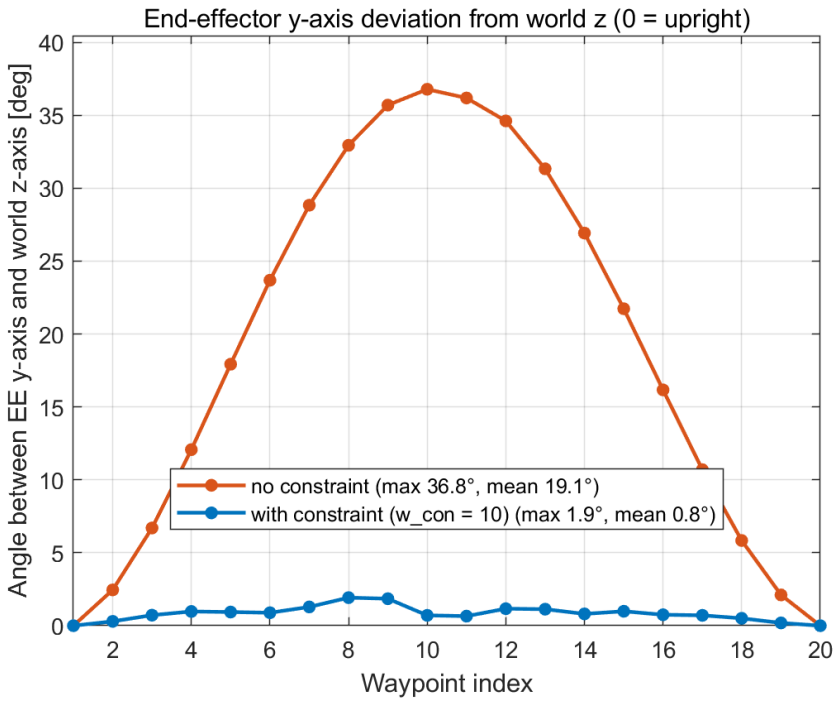


Figure 8. Angle between the gripper y-axis and the world z-axis at every waypoint, without and with the constraint.

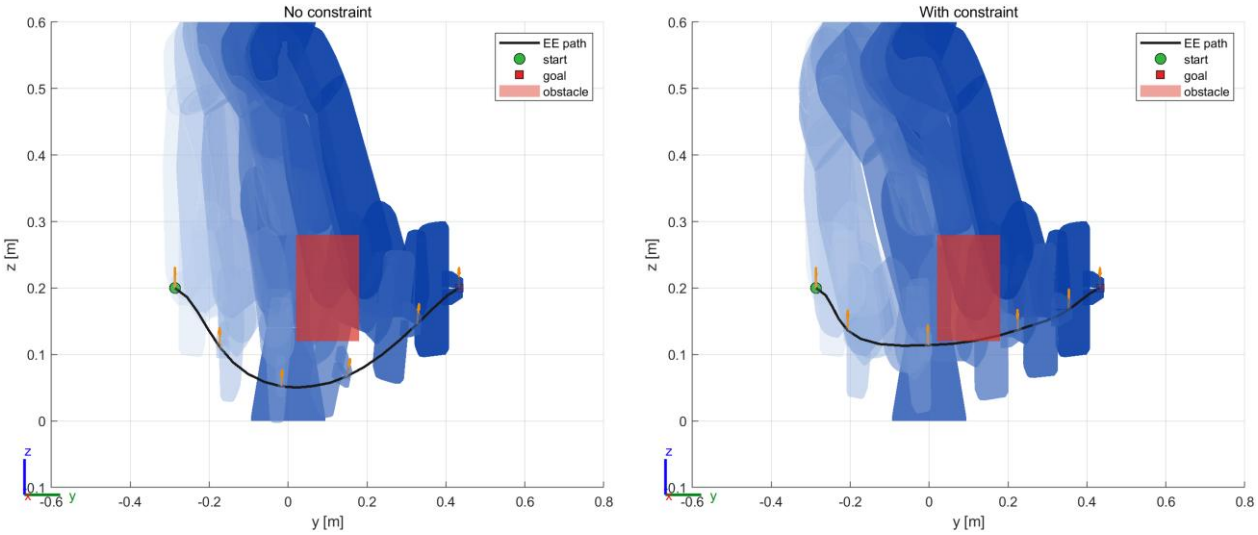


Figure 9. Task 5 trajectories without (left) and with (right) the constraint, side view (y–z plane); orange arrows: gripper y-axis at every 4th waypoint. The tilt is quantified in Figure 8; 3D and top views are in the results folder.

8 Improvements and additional features

8.1 Bugs fixed in the provided code

- Samples outside the voxel world: `sub2ind` failed, the `try/catch` returned obstacle cost 0, so a sample that left the workspace looked like the best one. Indices are now clamped.
- Control cost: `sum(θRθᵀ, 'all')` added cross-joint terms; replaced by the trace ($\sum_j \theta_j^T R\theta_j$).
- `qc_cost = 0` inside the loop overwrote the constraint-cost vector with a scalar.
- The safety margin was hard-coded (0.05 m) while the main script's `safetyDistance` was never used.
- The training video created an empty file when disabled and indexed past the end of the animation.
- The zero-update stop test used `sum(…) == 0`, which positive and negative entries can satisfy.
- Kinova: the end-effector link was not among the FK frames, so the last link had no spheres.

- `wrapToPi` was applied to every joint — wrong for prismatic joints and for revolute joints whose limits reach beyond π (Panda joint 6 up to 3.75 rad); now only continuous joints are wrapped.
- Coincident frames (Panda link 1/2, 5/6) have length 0 with getTransform but ~1e-17 with PoE; `ceil` then changed the number of spheres between waypoints and the velocity term failed.
- An obstacle partly outside the voxel world made `sub2ind` fail in the voxelisation.

8.2 STOMP extensions: joint limits and sample reuse

The original STOMP ignores the joint limits: the noise and the update can push a joint beyond its limit, and the planned trajectory is then not executable. This happened in our scenes (up to 0.52 rad, Table 8). We project every noisy sample and every updated trajectory into the limits (the noise becomes the clamped sample minus the mean), in the spirit of the joint-limit filter of the ROS STOMP implementation [5]. Projection alone made the tightest scenes harder; reusing samples fixes this: the five cheapest noisy trajectories of the previous iteration take part in the next update. Their costs are already known, so reuse costs no extra FK. We also tried shrinking the exploration noise every iteration (0.98 per iteration); it made the hard scenes fail, because the noise becomes too small before the detour is found, so it is off by default.

Table 8. Ablation over 7 scenes (Tasks 1, 2, 4 S1–S3, 5 without / with constraint) × 5 seeds (bonus_ablation.csv; per-scene numbers in the CSV and Figure 10). All variants use convergence patience $P = 3$; $P = 1$, the original stopping rule, is compared in Table 9.

Variant	Collision-free runs	Worst limit violation [rad]	Median iter. T4 S1	Median iter. T4 S3	Median iter. T5 constr.
no extensions	35 / 35	0.52	64	79	174
joint limits	31 / 35	0.00	64	147	200
limits + reuse (default)	35 / 35	0.00	43	85	162
limits + noise decay	29 / 35	0.00	73	78	200
limits + reuse + decay	35 / 35	0.00	53	69	78

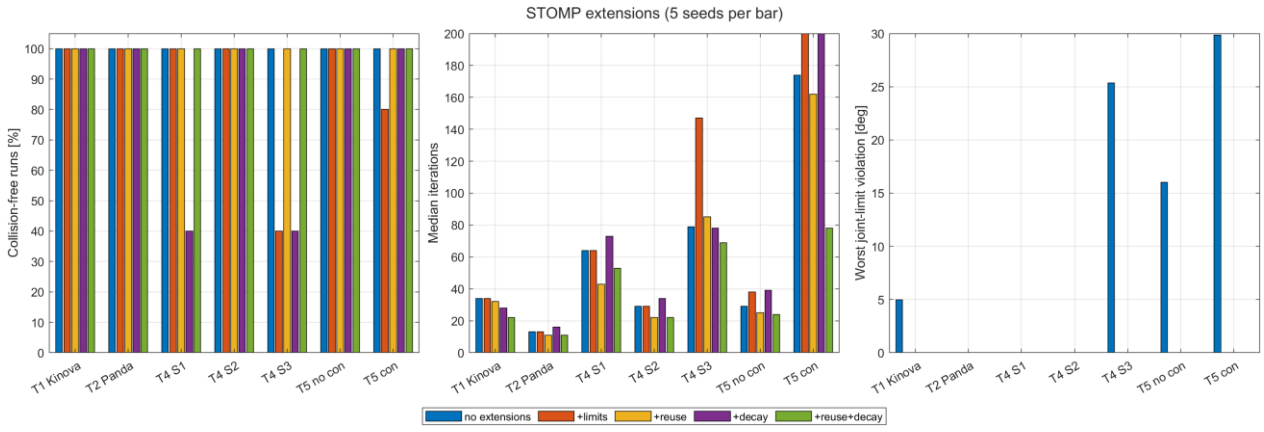


Figure 10. Ablation of the STOMP extensions: collision-free runs, median iterations and worst joint-limit violation.

8.3 Convergence patience

The provided planner stops as soon as a single iteration changes the total cost Q by less than 0.1. Because the updates of STOMP are stochastic, Q can briefly plateau before improving again, so this single-step test can stop the planner early. We instead require $|\Delta Q| < 0.1$ for P consecutive iterations, where $P = 1$ reproduces the original rule. Table 9 compares both on every project scene with five seeds (`compare_patience_all.m`). With $P = 1$, STOMP often stopped while the path still collided (4/5 in Task 4 S1; 1/5 in Task 4 S3; 4/5 in Task 5, with constraint collision-free). With $P = 3$, 35 of 35 runs were collision-free and on every Panda scene the median minimum mesh clearance increased (by 0.8–8.7 cm). The price is more iterations, from 9 % more (Task 5, no constraint) to about 6 times as many in the hardest case (Task 5, with constraint, median 28 → 173). In Task 1 the minimum clearance stays at 0.69 cm because it is limited by the provided goal pose, which STOMP cannot move.

Table 9. Convergence patience $P = 1$ (original stopping rule) vs $P = 3$ on every project scene, 5 seeds each (PoE FK, default settings; patience_summary.csv). Collision-free = mesh check at and between the waypoints; clearance = minimum mesh distance to the obstacles.

Scene	P	Collision-free	Median iterations	Median obstacle cost	Median min. clearance [cm]
Task 1 (Kinova)	1	5 / 5	11	16.78	0.69
Task 1 (Kinova)	3	5 / 5	32	13.05	0.69
Task 2/3 (Panda)	1	5 / 5	8	0.75	6.17
Task 2/3 (Panda)	3	5 / 5	11	0.00	7.09
Task 4 S1	1	4 / 5	34	2.76	5.36
Task 4 S1	3	5 / 5	43	0.00	8.05
Task 4 S2	1	5 / 5	16	1.53	6.82
Task 4 S2	3	5 / 5	22	0.31	9.75
Task 4 S3	1	1 / 5	22	34.59	0.00
Task 4 S3	3	5 / 5	85	3.55	3.84
Task 5, no constraint	1	5 / 5	22	0.00	8.79
Task 5, no constraint	3	5 / 5	24	0.00	9.59
Task 5, with constraint	1	4 / 5	28	27.82	1.22
Task 5, with constraint	3	5 / 5	173	0.09	9.94

8.4 Joint-angle discontinuity at ±π

Continuous joints (Kinova joints 1, 3, 5, 7) are wrapped to $[-\pi, \pi]$ after IK, so a plain straight line in joint space can turn a joint the long way round. For continuous joints `initTrajectory` now interpolates along the shortest angle, $\mathrm{wrapToPi}(q_T - q_0)$. In a demonstration the Kinova base turns from 2.8 to −2.8 rad (the Task-1 configurations rotated behind the robot): the naive line turns it 5.6 rad through the front of the robot and STOMP keeps that detour (end-effector path 3.31 m); the shortest-angle line turns it 0.68 rad (path 0.79 m). Both are collision-free, but STOMP is a local optimiser and cannot repair a bad initial guess of this kind (Figure 11).

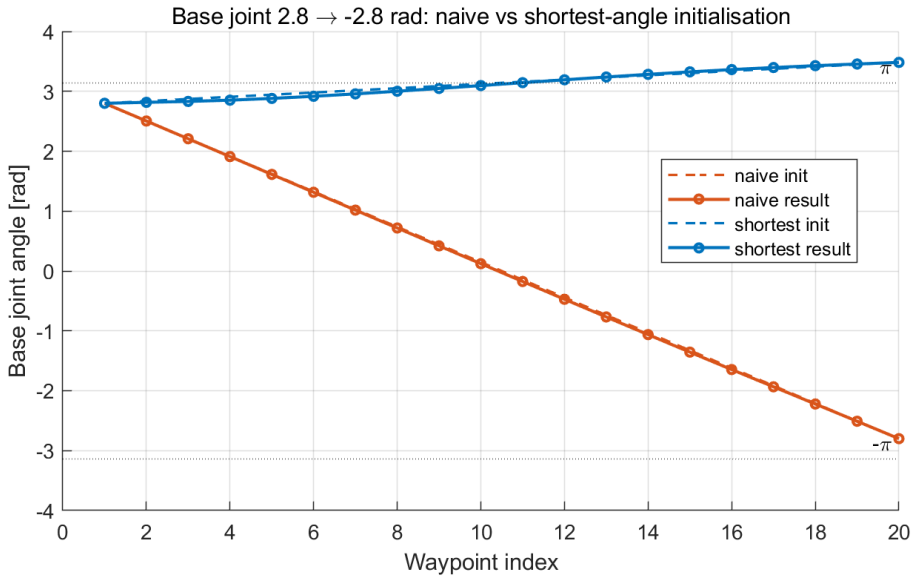


Figure 11. Base-joint angle along the trajectory, naive vs shortest-angle initialisation.

8.5 Further practical features

- **Efficiency:** PoE FK with a single running product (Section 5.2); getTransform without struct configurations; reused samples (Section 8.2).
- **Verification:** mesh-based collision check at the waypoints and between them; every final trajectory is also checked against the joint limits (all within limits).
- **Fitted collision spheres** (Section 4.3) instead of a hand-picked radius.
- **Robustness studies** with several seeds per scenario / weight instead of a single run.

- **Interactive GUI** (`GUI/stomp_gui.m`, Figure 12): choose a scenario, the FK method, the planner options, the seed and the constraint weight, plan, and watch the result with view presets and mouse rotation.
- **Reproducibility:** one code base, seeded runs, results and videos shipped in the zip; a clean-copy check runs every script from the unpacked submission.

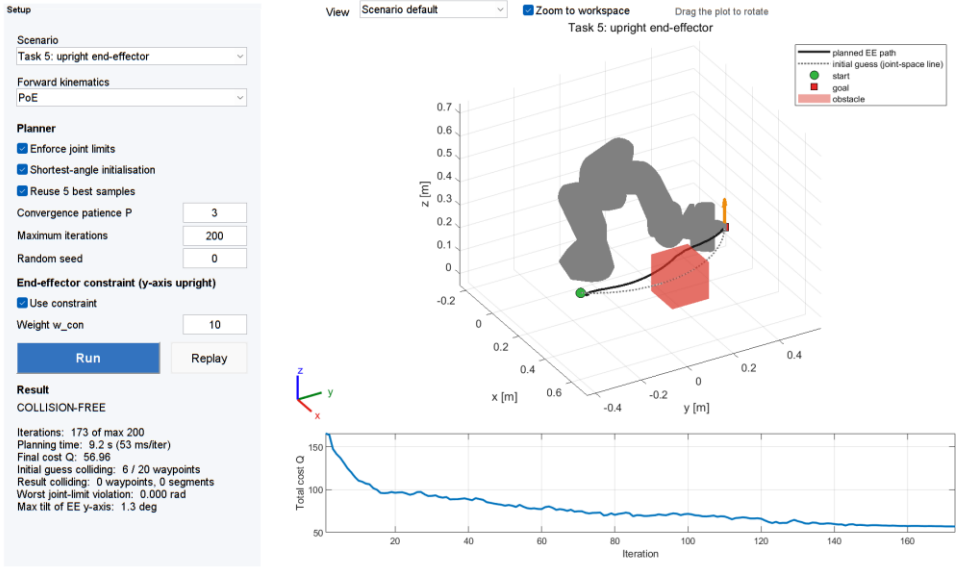


Figure 12. The GUI after planning Task 5 with the orientation constraint.

9 Limitations

- The sphere model contains 99 % of the mesh; the remaining corners (e.g. of the hand) are covered only by the 2 cm margin. The mesh-based check guarantees the final results, not the sphere model.
- The collision check between waypoints uses nine sub-steps per segment, not a continuous sweep.
- With larger constraint weights ($w_{\mathrm{con}} \geq 30$) the runs usually reach the iteration cap: the stop rule uses an absolute cost change, and the constraint keeps the cost fluctuating. At the chosen weight, 1 of 5 runs reach it.
- STOMP is a local, stochastic optimiser; success depends on the initial guess (Section 8.4) and the seed, which is why we report multi-seed success rates.
- Start and goal come from IK of fixed poses; an unreachable pose (Section 7.1) must be fixed in the scene.

10 Contributions

Every member implemented and presents their own part; the shared code was agreed in the README ("shared interfaces").

Table 10. Contributions of the group members.

Member	Contributions
Joshua Lim	Task 1: completion of the STOMP main loop and the incomplete functions (getTransform FK, sampling, $\delta\theta$, obstacle cost) and the related bug fixes. Task 2: choice of the Panda, robot model with virtual TCP, Task-2 scene. Framework (runPipeline, configuration). Convergence-patience evaluation.
Hu Jiamo	Task 3: screw axes (two methods), PoE FK, accuracy/speed/planning comparison, screw-axis figure. Task 4: multi-obstacle voxelisation, three scenarios, multi-seed robustness study. STOMP extensions and ablation, joint-angle discontinuity, fitted collision spheres, between-waypoint collision check, re-tuning of the Task-5 scene for the Panda, GUI, videos, packaging and clean-copy reproduction check.
Dexter	Task 5: orientation-constraint cost, upright start/goal scene, weight sweep and automatic choice, before/after analysis, EE-axis deviation plot and multi-view overlays. Shared plotting functions (configuration overlay, cost history). Report and slides integration.

11 AI declaration

AI (Claude) is used in our project to aid in the creation of the code, report and presentation.

References

[1] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in Proc. IEEE ICRA, 2011, pp. 4569–4574.

[2] K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control. Cambridge University Press, 2017.

[3] The MathWorks, Robotics System Toolbox documentation: loadrobot, getTransform, inverseKinematics, checkCollision, R2023b.

[4] Franka Emika, franka_ros 0.6.0 robot description (frankaEmikaPanda URDF shipped with MATLAB).

[5] ROS-Industrial, stomp_ros / stomp_moveit: STOMP implementation for MoveIt, including a joint-limits noise filter.